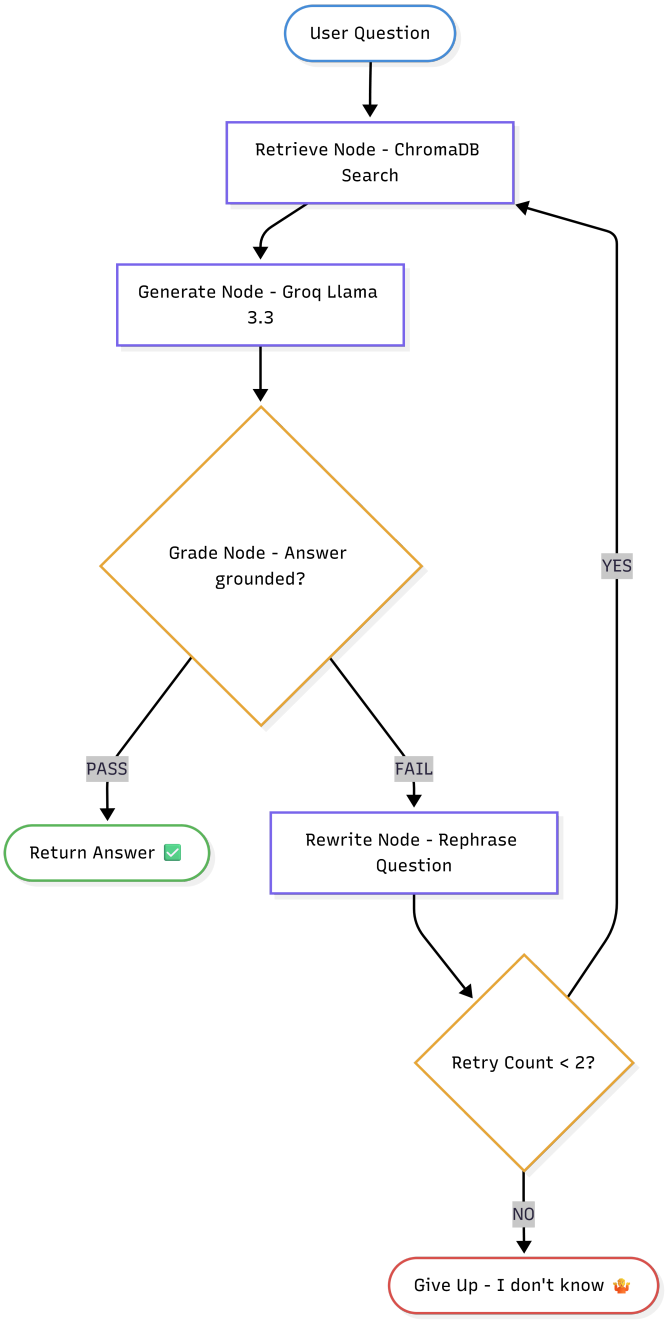
Build a Self-Healing RAG System in 60 Minutes

Created by | https://www.instagram.com/datasciencebrain



What Problem Does This Solve?

Have you ever built a chatbot that confidently gives a completely wrong answer? That is the number one problem with basic RAG (Retrieval-Augmented Generation) systems - they retrieve documents, pass them to an LLM, and just *hope* the answer is good. No checking. No fallback. No quality control.

A **Self-Healing RAG** system is smarter. After retrieving documents and generating an answer, it **grades its own output**. If the answer is bad - hallucinated, off-topic, or not supported by the retrieved documents - the system *automatically* rewrites the question and tries again. It heals itself before the user ever sees a wrong answer.

Who uses this in real life? Any company building a knowledge base chatbot over internal documents - HR policies, product manuals, legal contracts. They cannot afford an AI that confidently makes things up.

What You Will Build

A Python command-line app that:

1. Loads a set of text documents into a local vector database (ChromaDB)
2. Takes a user question and retrieves the most relevant document chunks
3. Generates an answer using Groq's free Llama 3.3 model
4. Automatically grades the answer - is it actually supported by the documents?
5. If the answer fails the grade, the system rewrites the question and tries again (up to 2 retries)
6. If all retries fail, returns an honest "I don't know" instead of hallucinating
7. The entire retry loop is orchestrated by LangGraph

Here is the flow in plain English:

```
User Question
      |
      ▼
[Retrieve Documents from ChromaDB]
      |
      ▼
```

```
[Generate Answer with Groq LLM]
      |
      ▼
[Grade Answer - Is it grounded in the docs?]
      |
   PASS ──────────────────────► Return Answer ✅
      |
   FAIL
      |
      ▼
[Rewrite Question - Try a different angle]
      |
      ▼
[Retrieve Again → Generate → Grade]   (max 2 retries)
      |
   STILL FAILING?
      |
      ▼
Return honest "I don't know" message 🤷
```

Estimated time: 60 minutes
Cost: ₹0 - all free APIs

Project Folder Structure

Here is what your finished project will look like. Read through this before writing any code - it helps to know where everything lives.

```
self-healing-rag/
├── .env                  ← your API keys (never commit this)
├── .gitignore            ← keeps secrets out of Git
├── pyproject.toml        ← auto-created by uv, lists dependencies
├── uv.lock               ← exact package versions (commit this!)
├── docs/                 ← your knowledge base text files
│   ├── python_basics.txt
│   └── machine_learning.txt
├── ingest.py             ← loads docs into ChromaDB
└── rag_agent.py          ← the self-healing LangGraph agent
```

Part 0 - Setup (10 minutes)

What is `uv` and why use it?

`uv` is a Python package manager written in Rust. It replaces both `pip` and `venv` in a single tool. Installing packages with `uv` is 10 to 100 times faster than pip, and it automatically manages your virtual environment for you. You never have to remember to "activate" anything - `uv run` handles it every time.

Step 1 - Install uv

Mac / Linux:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows (PowerShell):

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/
uv/install.ps1 | iex"
```

Close and reopen your terminal after installation, then verify it worked:

```
uv --version
```

You should see something like `uv 0.6.x`. If you do, you are ready.

Step 2 - Create the project

```
uv init self-healing-rag
cd self-healing-rag
```

`uv init` creates a clean project folder with a `pyproject.toml` file. This file is your project's identity card - it records the project name, required Python version, and every dependency you install.

Step 3 - Install all dependencies

```
uv add langgraph langchain-groq langchain-community langchain
-text-splitters chromadb python-dotenv
```

This single command installs everything. Here is what each package does:

Package	What it does
`langgraph`	Orchestrates our retrieve → grade → retry loop as a graph
`langchain-groq`	Connects LangChain to Groq's free LLM API
`langchain-community`	Provides document loaders for reading `.txt` and other files
`langchain-text-splitters`	Splits large documents into smaller chunks for better retrieval
`chromadb`	Our local vector database - stores and searches documents
`python-dotenv`	Reads API keys from a `.env` file at runtime

> ⚠️ **Important import note:** In older LangChain versions, text splitters lived inside the main `langchain` package. They were moved to a separate package. The correct import is `from langchain_text_splitters import ...` - if you use `from langchain.text_splitter import ...` you will get an ImportError.

Step 4 - Get your free Groq API key

1. Go to console.groq.com
2. Sign up with Google or email - completely free
3. Click **API Keys** in the left sidebar
4. Click **Create API Key**, give it a name, and copy the key

Groq's free tier gives you access to Llama 3.3 70B with generous rate limits - more than enough for this project and production prototypes.

Step 5 - Create your `.env` file

```
echo 'GROQ_API_KEY=your_key_here' > .env
```

Open the `.env` file in any text editor and replace `your_key_here` with the actual key you copied from Groq.

Step 6 - Protect your secrets with `.gitignore`

```
echo ".env" >> .gitignore
echo ".venv/" >> .gitignore
```

```
echo "__pycache__/" >> .gitignore
echo "chroma_db/" >> .gitignore
```

> ⚠️ **Common mistake:** Skipping this step, pushing to GitHub, and having API keys scraped by bots within minutes. Those bots are real and automated. Always add `.env` to `.gitignore` before your first commit.

Step 7 - Create the knowledge base documents

Create the `docs/` folder and add two sample files:

```
mkdir docs
```

Create `docs/python_basics.txt` and paste this content:

```
Python is a high-level, interpreted programming language created by Guido van Rossum in 1
991.
Python uses indentation to define code blocks instead of curly braces.
Python supports multiple programming paradigms including procedural, object-oriented, and
functional programming.
The Python Package Index (PyPI) hosts over 400,000 packages that extend Python's capabili
ties.
Python is widely used in data science, web development, automation, and artificial intell
igence.
Python's philosophy emphasizes code readability and simplicity, summarized in the Zen of
Python.
```

Create `docs/machine_learning.txt` and paste this content:

```
Machine learning is a subset of artificial intelligence that enables systems to learn fro
m data.
Supervised learning uses labeled training data to teach models to make predictions on new
examples.
Unsupervised learning finds hidden patterns in data without any labeled examples.
Neural networks are computational models inspired by the structure of the human brain.
Overfitting occurs when a model learns training data too well and performs poorly on new
unseen data.
Regularization techniques like dropout and L2 penalty help prevent overfitting in neural
networks.
The training process involves adjusting model weights to minimize a loss function over ma
ny iterations.
```

✅ **Checkpoint:** Run `ls docs/` - you should see both `.txt` files listed.

Part 1 - Understanding RAG: The Core Idea (5 minutes)

Before writing the ingestion code, let us understand what RAG actually does and why each part exists.

RAG stands for Retrieval-Augmented Generation. Here is the problem it solves:

LLMs like Llama 3.3 are trained on public internet data up to a certain date. They have never seen your company's private documents. If you ask "what does our HR policy say about sick leave?" the model cannot answer - it has never seen your HR document.

RAG fixes this by giving the LLM access to your documents at question time:

1. **Store your documents** as searchable vectors in a database before any questions arrive
2. **When a question arrives,** find the most relevant document chunks using similarity search
3. **Send those chunks along with the question** to the LLM as context
4. The LLM generates an answer based on *your documents*, not just its training data

Think of it as the difference between a closed-book exam and an open-book exam. Basic LLMs do closed-book. RAG gives the model the textbook.

What is a Vector Database?

A vector database (like ChromaDB) stores documents as lists of numbers called **embeddings**. An embedding captures the *meaning* of a piece of text as coordinates in high-dimensional space. The word "dog" and the word "puppy" end up at nearby coordinates even though they are spelled differently.

When you search with a question, the database converts your question into an embedding and returns the documents whose embeddings are *closest* - meaning most similar in meaning, not just matching exact keywords.

> 💡 **Why this matters:** Traditional search finds documents containing your exact search words. Vector search finds documents that *mean the same thing* as your question. This is dramatically better for question answering, especially when users phrase things differently from how the documents are written.

What is an Embedding Model?

An embedding model is a small neural network trained to convert text into these coordinate vectors. In this project we use ChromaDB's built-in default embedding model (`all-MiniLM-L6-v2` from SentenceTransformers). It is about 25MB, runs on your CPU, downloads automatically the first time you run the code, and costs absolutely nothing to use.

Part 2 - Building the Document Ingestion Pipeline (10 minutes)

The ingestion pipeline runs once (or whenever you add new documents). It reads your text files, splits them into manageable chunks, and stores them in ChromaDB.

Create a new file called `ingest.py`.

Step 1 - Imports

This block brings in every tool we need for loading and storing documents:

```
import os
from pathlib import Path
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSp
litter
import chromadb
from dotenv import load_dotenv
```

`Path` is Python's built-in tool for working with file paths in a way that works on both Windows and Mac/Linux. `TextLoader` reads `.txt` files. `RecursiveCharacterTextSplitter` breaks long text into chunks. `chromadb` is our vector database client.

Step 2 - Initialize ChromaDB

This creates (or reopens) a persistent ChromaDB database saved to your hard drive:

```
load_dotenv()

chroma_client = chromadb.PersistentClient(path="./chroma_db")

collection = chroma_client.get_or_create_collection(
    name="knowledge_base",
    metadata={"hnsw:space": "cosine"}
)
```

`PersistentClient` saves your vector data to the `chroma_db/` folder so you only need to run ingestion once. Every time after that, the data is already there.
`get_or_create_collection` is safe to call multiple times - if the collection already exists it opens it, otherwise it creates it.

> 💡 **Why `cosine` similarity?** For text, we care about the *direction* of a vector (what meaning does it point toward?) rather than its raw magnitude. Cosine similarity measures the angle between two vectors. An angle near 0° means very similar meaning. This works much better for text than measuring raw Euclidean distance.

Step 3 - Load and split the documents

This function reads every `.txt` file in the `docs/` folder and splits each one into small chunks:

```
def load_and_split_documents(docs_folder: str) -> list:
    """Load all .txt files from a folder and split into chunk
s."""
    all_chunks = []

    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=300,
```

```
        chunk_overlap=50,
        length_function=len,
    )

    docs_path = Path(docs_folder)
    for txt_file in docs_path.glob("*.txt"):
        loader = TextLoader(str(txt_file), encoding="utf-8")
        documents = loader.load()
        chunks = text_splitter.split_documents(documents)
        all_chunks.extend(chunks)
        print(f"  Loaded {len(chunks)} chunks from {txt_file.
name}")

    return all_chunks
```

`chunk_size=300` means each piece of text is at most 300 characters long. `chunk_overlap=50` means each chunk shares 50 characters with the one before it - this prevents cutting a sentence in half and losing the context needed to understand it.

> ⚠️ **Common mistake:** Setting `chunk_size` too large (like 2000 characters). Bigger chunks include more irrelevant content alongside the relevant content, which confuses the LLM. For precise question answering, smaller focused chunks (200 to 500 characters) consistently perform better.

> 💡 **Why `RecursiveCharacterTextSplitter`?** It tries to split on natural boundaries in order - paragraphs first, then sentences, then words. Only if none of those natural splits produce chunks small enough does it split mid-sentence. This keeps more meaningful units together compared to splitting every N characters blindly.

Step 4 - Store the chunks in ChromaDB

This function takes the processed chunks and writes them to the database:

```
def ingest_to_chromadb(chunks: list) -> None:
    """Store document chunks in ChromaDB."""
```

```
    documents = [chunk.page_content for chunk in chunks]
    ids = [f"chunk_{i}" for i in range(len(chunks))]
    metadatas = [
        {"source": chunk.metadata.get("source", "unknown")}
        for chunk in chunks
    ]

    collection.add(
        documents=documents,
        ids=ids,
        metadatas=metadatas
    )
    print(f"  Stored {len(documents)} chunks in ChromaDB.")
```

We pass three things to `collection.add()`: the text content (`documents`), a unique identifier for each chunk (`ids`), and metadata about where each chunk came from (`metadatas`). ChromaDB automatically converts the text into embeddings using its built-in SentenceTransformer model - no embedding API call, no API key, no cost.

Step 5 - The main runner

Add this block at the bottom of `ingest.py` to tie everything together:

```
if __name__ == "__main__":
    print("Starting document ingestion...")
    chunks = load_and_split_documents("docs")

    if not chunks:
        print("No .txt files found in docs/ folder.")
    else:
        ingest_to_chromadb(chunks)
        print(f"\nDone! Total chunks stored: {len(chunks)}")
        print("Your knowledge base is ready.")
```

Now run the ingestion:

```
uv run ingest.py
```

✅ **Checkpoint:** You should see output like this:

```
Starting document ingestion...
  Loaded 3 chunks from python_basics.txt
  Loaded 4 chunks from machine_learning.txt
  Stored 7 chunks in ChromaDB.

Done! Total chunks stored: 7
Your knowledge base is ready.
```

A `chroma_db/` folder should now exist in your project directory. That is your vector database - it contains your document chunks as searchable embeddings.

Part 3 - Understanding LangGraph (5 minutes)

Before writing the agent, let us understand LangGraph and why we are using it instead of plain Python.

LangGraph lets you build AI workflows as a graph. A graph has two things: **nodes** and **edges**. Each node is a Python function that does one job. Each edge is a connection that says "after this node finishes, go to that node next."

What makes LangGraph special compared to a normal function chain is that graphs can have **cycles** - a node can loop back to an earlier node. This is exactly what our retry pattern needs: after grading an answer as bad, we loop back to the retrieve node and try again.

Here is our graph in visual form:

```
[retrieve] → [generate] → [grade_answer]
                                 |
                           PASS → END
                                 |
                           FAIL → [rewrite_question] → [retrieve] (loops back)
                                 |
                     TOO MANY RETRIES → [give_up] → END
```

What is State?

LangGraph uses a **shared state dictionary** that every node can read from and write to. Think of it as a whiteboard in the middle of the room. When `retrieve` finishes, it writes the retrieved documents onto the whiteboard. When `generate` runs, it reads those documents from the whiteboard, generates an answer, and writes the answer back. When `grade_answer` runs, it reads the answer and writes whether it passed or failed.

This is how information flows between nodes without passing function arguments manually.

What is a Conditional Edge?

A regular edge always goes from node A to node B. A **conditional edge** looks at the current state and decides which node to go to next. After `grade_answer`, the conditional edge checks: did the answer pass? If yes, go to END. If no, go to `rewrite_question`. This is the routing logic at the heart of self-healing.

> 💡 **Why LangGraph instead of a plain Python loop?** You could write this as a `while` loop with `if/else`. LangGraph gives you: (1) clean separation between each step, (2) built-in state management so nodes don't need to pass data as arguments, (3) easy to add human-in-the-loop interrupts later, and (4) the graph structure makes the retry logic readable and maintainable.

Part 4 - Building the LangGraph Agent (25 minutes)

Now we build the heart of the system. Create a new file called `rag_agent.py`.

Step 1 - Imports

```
import os
from typing import TypedDict, Annotated
from dotenv import load_dotenv
import chromadb
from langchain_groq import ChatGroq
from langchain_core.messages import HumanMessage, SystemMessa
ge
```

```
from langgraph.graph import StateGraph, END

load_dotenv()
```

`TypedDict` is a Python type hint tool that lets us define a dictionary with specific named keys and their expected types. We will use it to define our agent's state. `StateGraph` and `END` are the core LangGraph building blocks.

Step 2 - Define the Agent State

What are type hints?
Type hints are notes you add to your code that say "this variable is expected to be a string" or "this function returns an integer." Python does not enforce them at runtime - they are documentation for you and your editor. The format is `variable_name: type`.

This is the state dictionary that every node in our graph will share:

```
class AgentState(TypedDict):
    question: str           # the user's original question
    rewritten_question: str # a rephrased version for retry
    documents: list[str]    # chunks retrieved from ChromaDB
    answer: str             # the generated answer
    grade: str              # "pass" or "fail"
    retry_count: int        # how many retries have happened
```

Every key in `AgentState` is a slot on our shared whiteboard. When we define the graph with `StateGraph(AgentState)`, LangGraph knows exactly what shape the state should be and validates it as nodes write to it.

Step 3 - Initialize the LLM and Vector Store

Set up the two external dependencies our nodes will use:

```
llm = ChatGroq(
    model="llama-3.3-70b-versatile",
    temperature=0,
    api_key=os.environ.get("GROQ_API_KEY")
```

```
)

chroma_client = chromadb.PersistentClient(path="./chroma_db")
collection = chroma_client.get_or_create_collection(
    name="knowledge_base",
    metadata={"hnsw:space": "cosine"}
)
```

`temperature=0` tells the LLM to be as deterministic as possible - we want factual answers based on documents, not creative variations. We open the same `chroma_db/` folder we created during ingestion so we are reading from the same database.

> 💡 **Why `llama-3.3-70b-versatile`?** This is Groq's recommended general-purpose model as of April 2026. The "70b" means 70 billion parameters - large enough to reason well about document content and grade its own answers. It runs on Groq's LPU hardware and returns responses in under a second on most queries.

Step 4 - The Retrieve Node

The retrieve node takes the current question from state, searches ChromaDB, and returns the top matching document chunks:

```
def retrieve(state: AgentState) -> AgentState:
    """Search ChromaDB for documents relevant to the current
question."""
    question = state.get("rewritten_question") or state["ques
tion"]

    print(f"\n[RETRIEVE] Searching for: {question}")

    try:
        results = collection.query(
            query_texts=[question],
            n_results=3
        )
```

```
        documents = results["documents"][0]
        print(f"[RETRIEVE] Found {len(documents)} chunks.")
    except Exception as error:
        print(f"[RETRIEVE] ChromaDB query failed: {error}")
        documents = []

    return {**state, "documents": documents}
```

`state.get("rewritten_question") or state["question"]` means: use the rewritten question if one exists (we are on a retry), otherwise use the original question. `n_results=3` retrieves the top 3 most similar chunks - enough context without overwhelming the LLM.

The `{**state, "documents": documents}` syntax creates a new dictionary with all the existing state fields plus the updated `documents` key. In LangGraph, nodes return the updated state.

> ⚠️ **Common mistake:** Forgetting the `try/except` block around external calls. If ChromaDB has an issue (the database is corrupted, a file is locked, disk is full), your agent will crash with an unhelpful traceback. The try/except lets you print a useful message and continue gracefully.

Step 5 - The Generate Node

The generate node takes the retrieved documents and question, formats them into a prompt, and asks Groq to produce an answer:

```
def generate(state: AgentState) -> AgentState:
    """Generate an answer using retrieved documents as contex
t."""
    question = state.get("rewritten_question") or state["ques
tion"]
    documents = state.get("documents", [])

    print(f"\n[GENERATE] Generating answer for: {question}")

    if not documents:
```

```
        return {**state, "answer": "No relevant documents were found."}

    context = "\n\n".join([f"Document {i+1}:\n{doc}"
                           for i, doc in enumerate(documents)])

    system_prompt = """You are a helpful assistant that answers questions
based strictly on the provided documents. If the documents do not contain
enough information to answer the question, say so honestly.
Do not use any knowledge outside of the provided documents."""

    user_prompt = f"""Documents:
{context}

Question: {question}

Answer based only on the documents above:"""

    try:
        response = llm.invoke([
            SystemMessage(content=system_prompt),
            HumanMessage(content=user_prompt)
        ])
        answer = response.content
        print(f"[GENERATE] Answer generated ({len(answer)} chars).")
    except Exception as error:
        print(f"[GENERATE] LLM call failed: {error}")
        answer = "An error occurred while generating the answer."
```

```
    return {**state, "answer": answer}
```

Notice the system prompt tells the LLM to answer *strictly* from the provided documents. This is critical for preventing hallucination - without this instruction, the model will happily blend document content with its own training knowledge and you cannot tell which is which.

> 💡 **Why join documents with `\n\n`?** Each retrieved chunk is a separate string. We join them with double newlines so the LLM can clearly see where one document ends and the next begins. Clear formatting in prompts leads to significantly better answers.

Step 6 - The Grade Node

This is the key node that makes the system "self-healing." It asks the LLM to evaluate whether the answer it just gave is actually supported by the retrieved documents:

```
def grade_answer(state: AgentState) -> AgentState:
    """Grade whether the answer is grounded in the retrieved
documents."""
    question = state["question"]
    documents = state.get("documents", [])
    answer = state.get("answer", "")

    print(f"\n[GRADE] Evaluating answer quality...")

    context = "\n\n".join(documents) if documents else "No do
cuments."

    grading_prompt = f"""You are a strict grader evaluating w
hether an answer
is grounded in the provided documents.

Documents:
{context}
```

```
Question: {question}

Answer: {answer}

Evaluate this answer on two criteria:
1. Is the answer directly supported by information in the doc
uments?
2. Does the answer avoid making claims not found in the docum
ents?

Respond with ONLY one word: PASS or FAIL.
PASS means the answer is well-supported by the documents.
FAIL means the answer contains unsupported claims or is not r
elevant."""

    try:
        response = llm.invoke([HumanMessage(content=grading_p
rompt)])
        grade_raw = response.content.strip().upper()
        grade = "pass" if "PASS" in grade_raw else "fail"
        print(f"[GRADE] Result: {grade.upper()}")
    except Exception as error:
        print(f"[GRADE] Grading call failed: {error}")
        grade = "fail"

    return {**state, "grade": grade}
```

We are using the LLM to grade itself. This pattern is called **LLM-as-judge** and it works surprisingly well. The key is the strict grading prompt - "respond with ONLY one word: PASS or FAIL" keeps the output parseable. The `.strip().upper()` and `"PASS" in grade_raw` check handles any stray whitespace or extra words the model might add.

> 💡 **Why does self-grading work?** The LLM can compare two pieces of text (the answer vs the documents) and judge whether one is supported by the other.

> This comparison task is easier and more reliable than generating an accurate answer in the first place - which is exactly why grading improves overall quality.

Step 7 - The Rewrite Node

When the grade fails, this node rephrases the original question to approach the retrieval from a different angle:

```
def rewrite_question(state: AgentState) -> AgentState:
    """Rewrite the question to improve retrieval on the next
attempt."""
    original_question = state["question"]
    retry_count = state.get("retry_count", 0)

    print(f"\n[REWRITE] Rewriting question (attempt {retry_co
unt + 1})...")

    rewrite_prompt = f"""The following question did not retri
eve useful documents
from a knowledge base. Rewrite the question to be more specif
ic and use
different vocabulary that might match relevant documents bett
er.

Original question: {original_question}

Write only the rewritten question, nothing else:"""

    try:
        response = llm.invoke([HumanMessage(content=rewrite_p
rompt)])
        rewritten = response.content.strip()
        print(f"[REWRITE] New question: {rewritten}")
    except Exception as error:
        print(f"[REWRITE] Rewrite failed: {error}")
        rewritten = original_question
```

```
    return {
        **state,
        "rewritten_question": rewritten,
        "retry_count": retry_count + 1
    }
```

The rewrite prompt gives the LLM context: "the previous retrieval didn't work, try different vocabulary." This is important - sometimes questions use synonyms that do not appear in documents. For example "Python programming language" might retrieve well while "Python coding" does not, depending on how your documents are worded.

We also increment `retry_count` here. This number is checked by the conditional edge to decide when to stop retrying.

Step 8 - The Give Up Node

When all retries are exhausted, this node returns an honest response instead of a hallucinated one:

```
def give_up(state: AgentState) -> AgentState:
    """Return an honest 'I don't know' after all retries are
exhausted."""
    print(f"\n[GIVE UP] Could not find a reliable answer afte
r retries.")

    honest_answer = (
        "I was unable to find a reliable answer to your quest
ion "
        "in the available documents. The knowledge base may n
ot contain "
        "information about this topic. Please try rephrasing
your question "
        "or consult additional sources."
    )
```

```
    return {**state, "answer": honest_answer}
```

This node exists for one reason: a system that says "I don't know" is infinitely more trustworthy than a system that confidently makes things up. Real users will forgive a "not found" response. They will not forgive a confident wrong answer that leads them to make a bad decision.

Step 9 - The Routing Function

This function is used as a conditional edge. It looks at the current state and returns a string telling LangGraph which node to go to next:

```
def route_after_grading(state: AgentState) -> str:
    """Decide where to go after grading the answer."""
    grade = state.get("grade", "fail")
    retry_count = state.get("retry_count", 0)
    max_retries = 2

    if grade == "pass":
        return "end"
    elif retry_count >= max_retries:
        return "give_up"
    else:
        return "rewrite"
```

The return values `"end"`, `"give_up"`, and `"rewrite"` are labels we will map to actual nodes in the next step. `max_retries = 2` means the system will try the original question once plus two rewritten versions before giving up.

Step 10 - Build and Compile the Graph

Now we wire all the nodes together:

```
def build_graph() -> StateGraph:
    """Assemble all nodes and edges into the LangGraph."""
    graph = StateGraph(AgentState)
```

```
    # Add all nodes
    graph.add_node("retrieve", retrieve)
    graph.add_node("generate", generate)
    graph.add_node("grade_answer", grade_answer)
    graph.add_node("rewrite_question", rewrite_question)
    graph.add_node("give_up", give_up)

    # Set the entry point - first node to run
    graph.set_entry_point("retrieve")

    # Add fixed edges (always go from A to B)
    graph.add_edge("retrieve", "generate")
    graph.add_edge("generate", "grade_answer")
    graph.add_edge("rewrite_question", "retrieve")  # the ret
ry loop!
    graph.add_edge("give_up", END)

    # Add the conditional edge after grading
    graph.add_conditional_edges(
        "grade_answer",
        route_after_grading,
        {
            "end": END,
            "give_up": "give_up",
            "rewrite": "rewrite_question"
        }
    )

    return graph.compile()
```

`add_conditional_edges` takes three arguments: the source node, the routing function, and a mapping from the routing function's return values to destination nodes. When `route_after_grading` returns `"rewrite"`, the edge goes to the `rewrite_question` node. When it returns `"end"`, it goes to the built-in `END` marker which stops the graph.

`graph.compile()` validates your graph (checks for disconnected nodes, missing entry points, etc.) and returns a runnable object.

Step 11 - The Main Runner

Add this to the bottom of `rag_agent.py` to create a simple command-line interface:

```
if __name__ == "__main__":
    print("Self-Healing RAG System")
    print("=" * 40)
    print("Type your question and press Enter.")
    print("Type 'quit' to exit.\n")

    rag_graph = build_graph()

    while True:
        user_question = input("Your question: ").strip()

        if user_question.lower() in ("quit", "exit", "q"):
            print("Goodbye!")
            break

        if not user_question:
            continue

        initial_state: AgentState = {
            "question": user_question,
            "rewritten_question": "",
            "documents": [],
            "answer": "",
            "grade": "",
            "retry_count": 0
        }

        try:
            final_state = rag_graph.invoke(initial_state)
```

```
            print(f"\n{'=' * 40}")
            print(f"ANSWER:\n{final_state['answer']}")
            print(f"{'=' * 40}\n")
        except Exception as error:
            print(f"An error occurred: {error}")
```

We create a fresh `initial_state` for every question - all fields start empty or at zero. `rag_graph.invoke(initial_state)` runs the graph to completion and returns the final state dictionary. The answer is in `final_state["answer"]`.

Part 5 - Run and Test (5 minutes)

Start the agent:

```
uv run rag_agent.py
```

You will see the startup banner and a prompt. Now run the three tests below.

Test 1 - Happy Path

```
Your question: What is Python used for?
```

What to watch: The `[RETRIEVE]` step should find relevant chunks from `python_basics.txt`. The `[GRADE]` step should return `PASS` on the first try. The answer should mention data science, web development, or AI.

Test 2 - A Trickier Question

```
Your question: Who made Python and when?
```

What to watch: The answer should correctly say Guido van Rossum in 1991. This information is in your documents. Watch whether it passes on the first try or needs a retry.

Test 3 - Question Not in the Knowledge Base

```
Your question: What is the capital of France?
```

What to watch: Your documents have nothing about geography. The system should retrieve documents, generate an answer the LLM makes up from its training knowledge (since the documents don't help), the grade node should detect this answer is not supported by the retrieved documents, and after retries the system should return the honest "I was unable to find a reliable answer" message.

> ✅ **This is the most important test.** If your system honestly says it does not know when the documents do not contain the answer, your Self-Healing RAG is working correctly.

Test 4 - Edge Case: Empty Question

Just press Enter without typing anything. The `while True` loop checks `if not user_question: continue` and skips gracefully without crashing.

Understanding the Full Flow in Action

Here is what happens inside the system for Test 3 step by step:

1. **retrieve:** Searches for "capital of France" in your documents. Finds the 3 most similar chunks - probably something about Python or ML that has low similarity but is still the "closest" ChromaDB has.
2. **generate:** Passes those irrelevant chunks plus the question to Llama 3.3. The LLM might say "Paris" (from its training data, ignoring the instruction to stick to documents) or say "the documents don't contain this information."
3. **grade_answer:** Asks the LLM: is this answer supported by the retrieved documents? The documents say nothing about France. Grade: `FAIL`.
4. **rewrite_question:** Rephrases to something like "What European city serves as the governmental center of France?" and increments retry_count to 1.
5. **retrieve:** Searches again with the new question. Still finds Python/ML chunks.
6. **generate → grade_answer:** Still fails. retry_count becomes 2.

7. **route_after_grading:** retry_count (2) >= max_retries (2), so routes to `give_up`.
8. **give_up:** Returns the honest "I was unable to find a reliable answer" message.

What to Build Next - 5 Upgrade Ideas

You now have a working Self-Healing RAG. Here is how to take it further:

1. Add a web search fallback with Tavily
When the local documents fail after all retries, instead of giving up, search the web. Install `uv add tavily-python`, get a free Tavily API key (1,000 searches/month free), and add a `web_search` node that fires right before `give_up`. The graph becomes: local retrieval fails → web search → generate from web results → grade → return.

2. Build a Streamlit chat interface
Turn the command-line app into a web chat UI. Run `uv add streamlit` and create `app.py`. Use `st.chat_message` and `st.chat_input` components. Pass conversation history as a list in the state so the system can handle follow-up questions like "tell me more about that."

3. Ingest PDFs instead of plain text
Most real knowledge bases are PDFs. Replace `TextLoader` with `PyPDFLoader` from `langchain-community`. Run `uv add pypdf`. Store the page number in metadata so your answer can cite "Source: document.pdf, page 4" - this dramatically increases user trust.

4. Add answer citations
Modify the `generate` node's system prompt to instruct the LLM: "Quote the exact phrase from the documents that supports your answer." Parse the quoted phrase and display it below the answer as a citation. Users can then verify answers themselves - essential for high-stakes use cases.

5. Upgrade to hybrid search
ChromaDB currently uses only semantic (vector) search. For technical documents with specific terminology, jargon, or product names, keyword search often beats semantic search. Add BM25 keyword search alongside vector search (ChromaDB supports this), then combine both result sets using Reciprocal Rank Fusion. This is what production RAG systems at large companies actually use.

Code Verification

All API calls in this guide were verified against current documentation as of April 2026:

Library	Version	API Call	Status
chromadb	1.5.7	`PersistentClient(path=...)`	✅ Verified
chromadb	1.5.7	`get_or_create_collection(name=..., metadata=...)`	✅ Verified
chromadb	1.5.7	`collection.add(documents=..., ids=..., metadatas=...)`	✅ Verified
chromadb	1.5.7	`collection.query(query_texts=..., n_results=...)`	✅ Verified
langchain-text-splitters	0.3.x	`from langchain_text_splitters import RecursiveCharacterTextSplitter`	✅ Verified
langchain-groq	0.3.x	`from langchain_groq import ChatGroq`	✅ Verified
langchain-groq	0.3.x	`ChatGroq(model="llama-3.3-70b-versatile", temperature=0)`	✅ Verified
langgraph	1.1.x	`StateGraph(AgentState)`	✅ Verified
langgraph	1.1.x	`graph.add_node(...)`	✅ Verified
langgraph	1.1.x	`graph.add_conditional_edges(...)`	✅ Verified
langgraph	1.1.x	`graph.compile()`	✅ Verified
langgraph	1.1.x	`compiled_graph.invoke(state)`	✅ Verified
Groq model	-	`llama-3.3-70b-versatile`	✅ Active as of April 2026
ChromaDB embeddings	-	Default SentenceTransformer `all-MiniLM-L6-v2`	✅ Free, no API key

> ⚠️ **Deprecated - do not use:** `from langchain.text_splitter import RecursiveCharacterTextSplitter` (old location, removed in recent LangChain versions). Always use `from langchain_text_splitters import ...`.

You just built what most enterprise AI teams take months to implement - a RAG system that knows when it's wrong and automatically fixes itself. That's not a

chatbot. That's a reasoning engine. Ship it. 🔥